

Systems Analysis and Design

Chapter 11: Software Reuse



Learning Objectives

11.1 Definition

11.2 COTS product reuse

11.3 Component development for reuse

11.4 Function reuse

Definition (1 of 2)

- In most engineering disciplines, systems are designed by composing existing components that have been used in other systems.
- Software engineering has been more focused on original development but it is now recognized that to achieve better software, more quickly and at lower cost, *we need to adopt a design process that is based on systematic software reuse.*



Definition (1 of 2)

- When you reuse program or design components, you have to follow the design decisions made by the original developer of the component
- This may limit the opportunities for reuse
- However, a more abstract form of reuse is concept reuse when a particular approach is described in an implementation independent way and an implementation is then developed
- The two main approaches to concept reuse are:
 - Design patterns;
 - Generative programming.

Design Patterns

- A design pattern is a way of reusing abstract knowledge about a problem and its solution.
- Description of the problem and the essence of its solution.
- Should be sufficiently abstract to be reused in different settings.
- Patterns often rely on object characteristics such as inheritance and polymorphism.

What is Design Pattern?



Generator-based Reuse

- Program generators involve the reuse of standard patterns and algorithms.
- These are embedded in the generator and parameterized by user commands. A program is then automatically generated.
- Generator-based reuse is possible when domain abstractions and their mapping to executable code can be identified.
- A domain specific language is used to compose and control these abstractions.

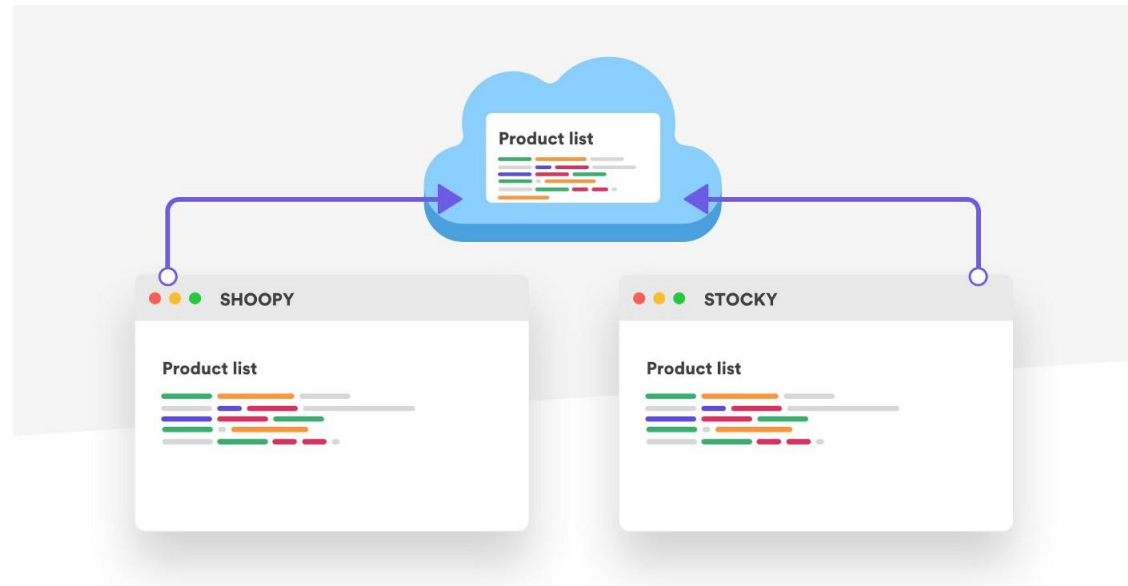
What can reuse? (1 of 3)

- Application system reuse
 - The whole of an application system may be reused either by incorporating it without change into other systems (COTS reuse) or by developing application families
- Component reuse
 - Components of an application from sub-systems to single objects may be reused
- Object and function reuse
 - Software components that implement a single well-defined object or function may be reused

What can reuse? (2 of 3)

Another understanding:

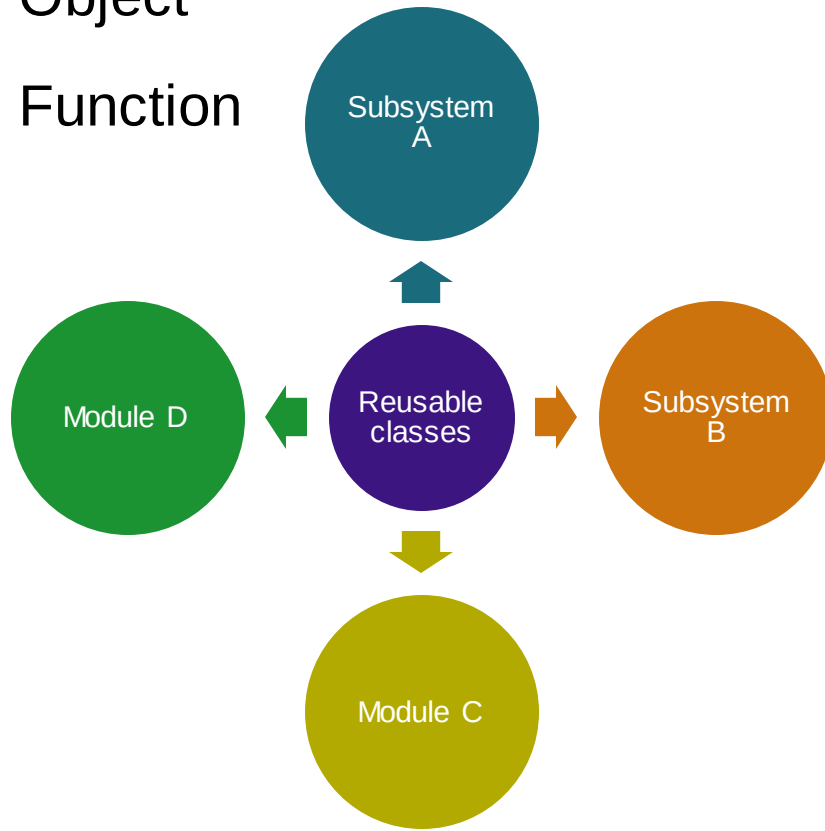
- Application system
- Subsystem
- Component
- Module
- Object
- Function or Procedure



What can reuse? (3 of 3)

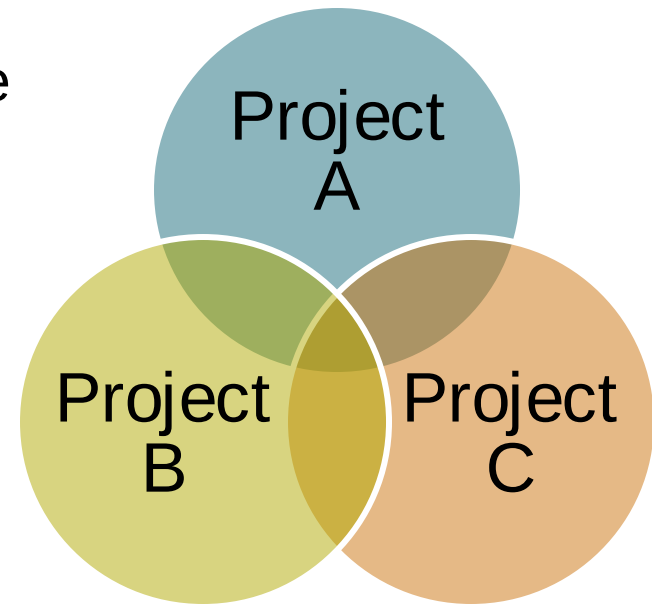
Reuse inside project:

- Module
- Object
- Function



In another projects:

- Application system
- Subsystem
- Component
- Module
- Object

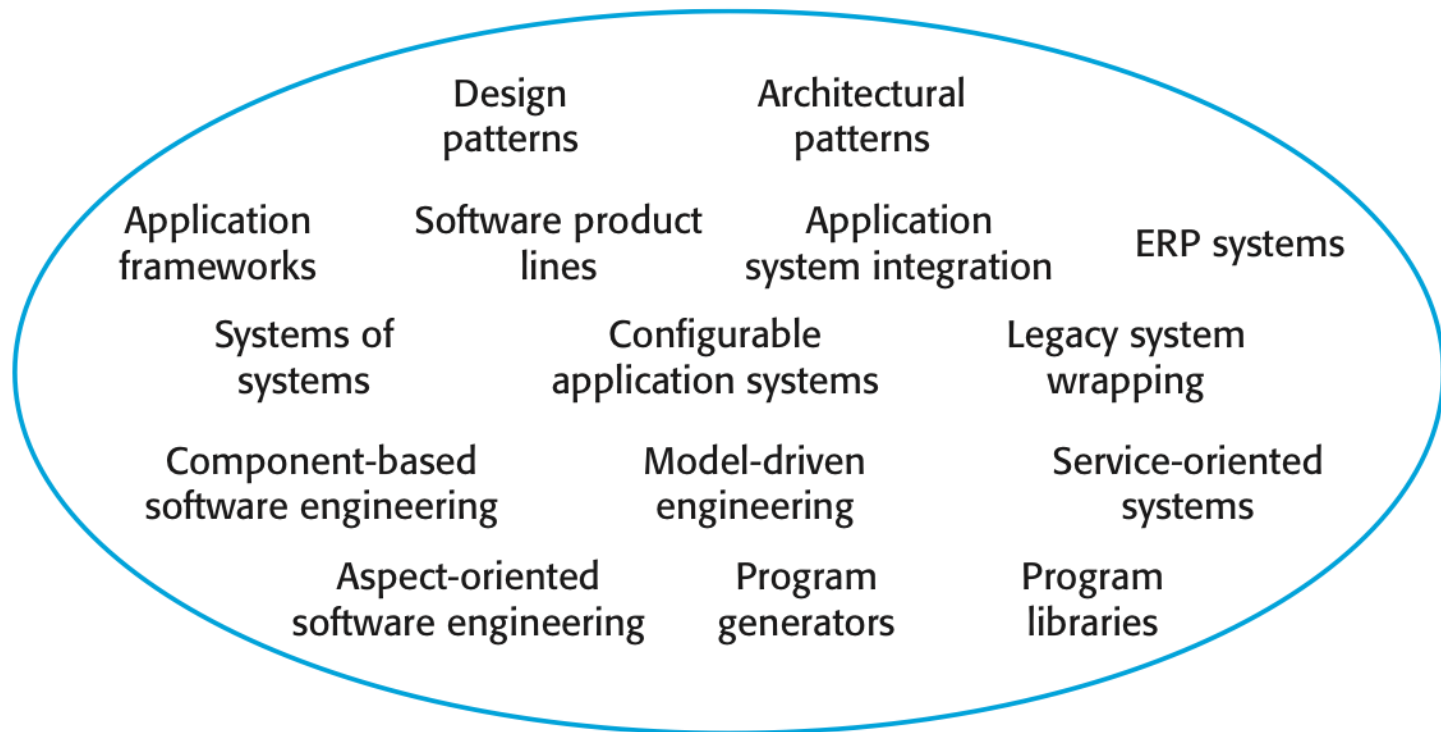


The Reuse Landscape (1 of 2)

- Although reuse is often simply thought of as the reuse of system components, there are many different approaches to reuse that may be used.
- Reuse is possible at a range of levels from simple functions to complete application systems.
- The reuse landscape covers the range of possible reuse techniques.

The Reuse Landscape (2 of 2)

- Techniques of Reuse Landscape



Reuse: Pros and Cons

Problems:

- Increased software (sw) maintenance costs
- Lack of tool support
- Not-invented-here syndrome
- Difficult in creating and maintaining a component library
- Finding and adapting reusable components

Benefits:

- Increase software productivity, interoperability, more standardized
- Shorten sw development time, develop sw with fewer people
- Move personnel more easily from project to project
- Reduce costs
- Produce better quality sw and provide a powerful competitive advantage

Reuse Planning Factors

- The development schedule for the software
- The expected software lifetime
- The background, skills and experience of the development team
- The criticality of the software and its non-functional requirements
- The application domain
- The execution platform for the software

Reuse in High Level Programming Languages

- Object codes (from different high-level programming languages) can be put together to make a library (*.lib*).
- Binary codes are reusable as libraries on computers of the same architecture (compile-time sharing).
- Libraries and object files on a computer are linked together to form an executable. (compile-time sharing of binary code).
- A dynamically-linked library (*.dll*) can be shared by all programs on the same computer and by all the running processes on the same computer (run-time sharing).
- Libraries (*.lib and .dll*) are machine-dependent

Learning Objectives

11.1 Definition

11.2 COTS product reuse

11.3 Component development for reuse

11.4 Function reuse

COTS Product Reuse

- COTS - Commercial Off-The-Shelf systems
- COTS systems are usually complete application systems that offer an API (Application Programming Interface)
- Building large systems by integrating COTS systems is now a viable development strategy for some types of system such as E-commerce systems

Don't Reinvent The Wheel !



COTS System Integration Problems

- Lack of control over functionality and performance
 - COTS systems may be less effective than they appear
- Problems with COTS system inter-operability
 - Different COTS systems may make different assumptions that means integration is difficult
- No control over system evolution
 - COTS vendors not system users control evolution
- Support from COTS vendors
 - COTS vendors may not offer support over the lifetime of the product

Learning Objectives

11.1 Definition

11.2 COTS product reuse

11.3 Component development for reuse

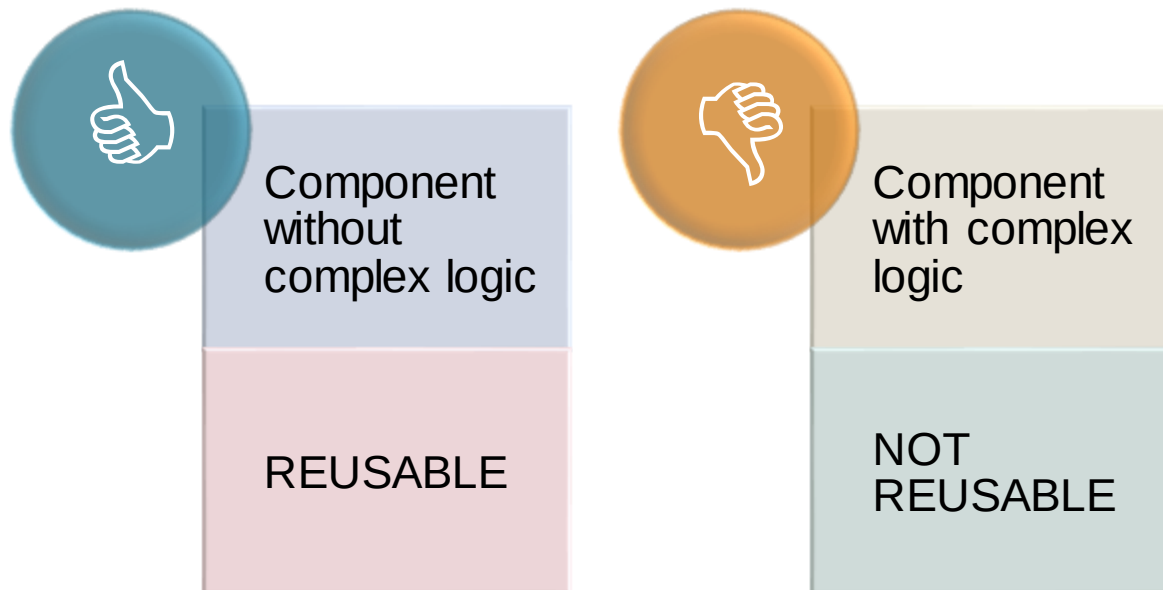
11.4 Function reuse

Component Development For Reuse

- Components for reuse may be specially constructed by generalizing existing components
- Component reusability
 - Should reflect stable domain abstractions
 - Should hide state representation
 - Should be as independent as possible
 - Should publish exceptions through the component interface
- There is a trade-off between reusability and usability.
- The more general the interface, the greater the reusability but it is then more complex and hence less usable

Reusable Components

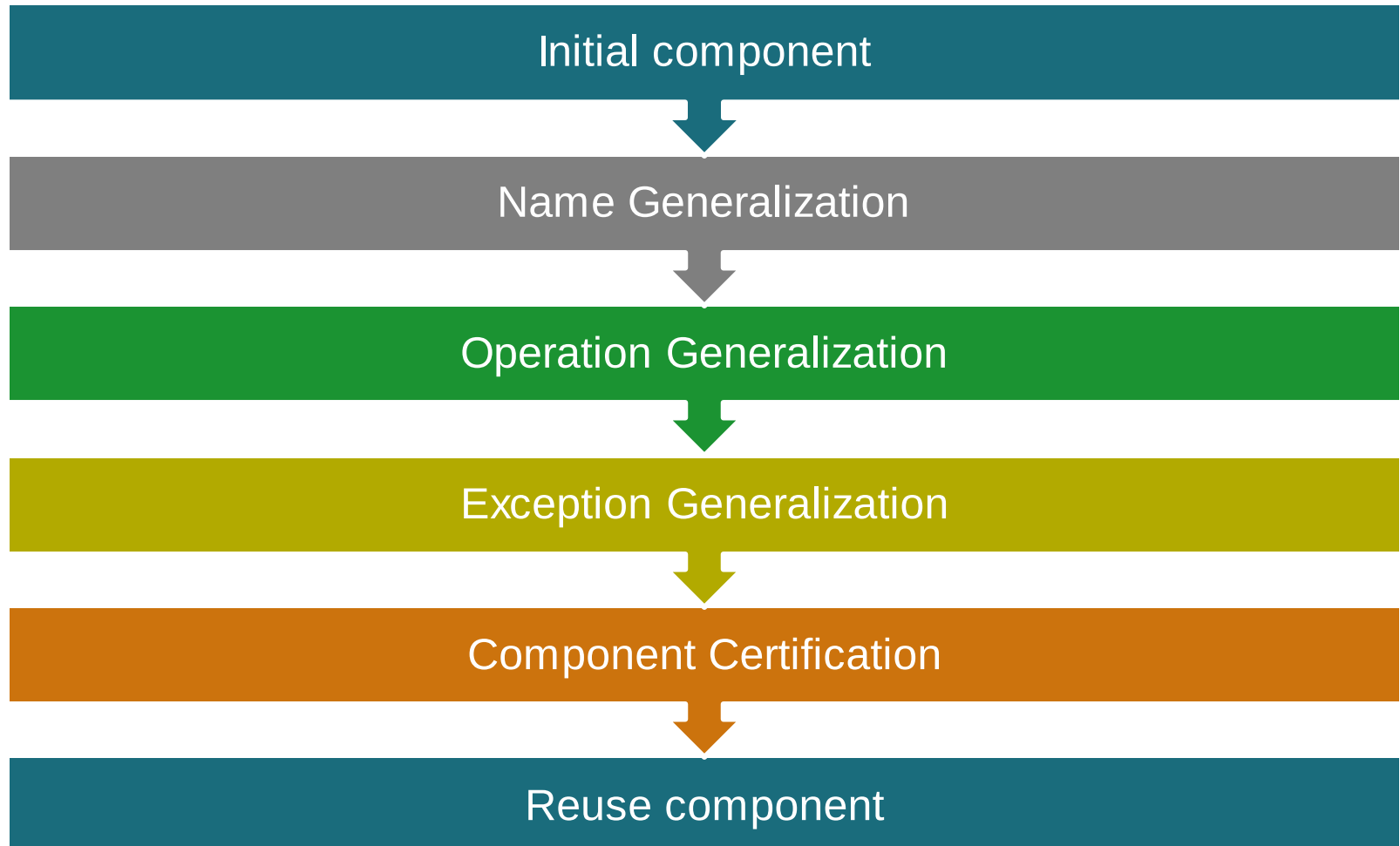
- The development cost of reusable components is higher than the cost of specific equivalents. This extra reusability enhancement cost should be an organization rather than a project cost
- Generic components may be less space-efficient and may have longer execution times than their specific equivalents



Reusability Enhancement

- Name generalization
 - Names in a component may be modified so that they are not a direct reflection of a specific application entity
- Operation generalization
 - Operations may be added to provide extra functionality and application specific operations may be removed
- Exception generalization
 - Application specific exceptions are removed and exception management added to increase the robustness of the component
- Component certification
 - Component is certified as reusable

Reusability Enhancement Process



Learning Objectives

11.1 Definition

11.2 COTS product reuse

11.3 Component development for reuse

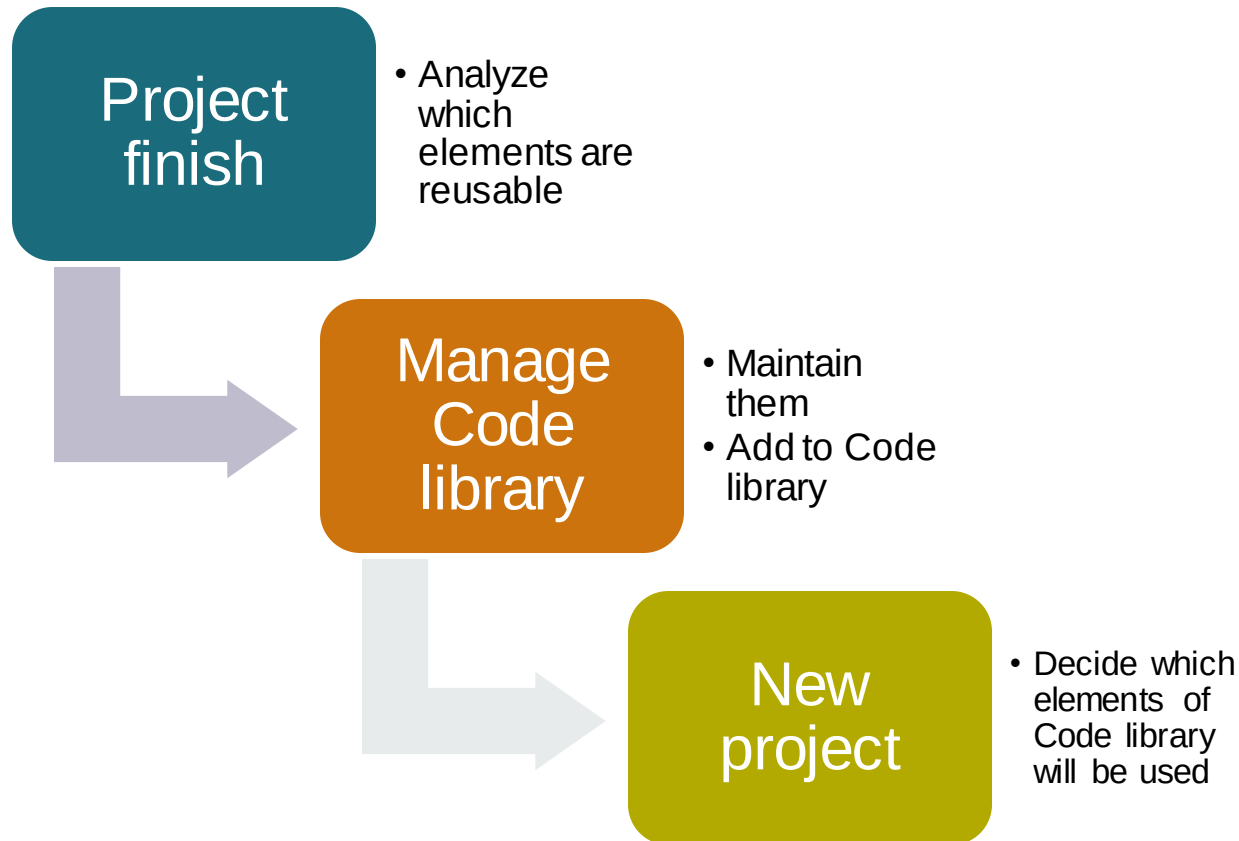
11.4 Function reuse

Object & Function Reuse

- Common technical functions which can be called from many others functions, e.g.:
 - Read/write Excel file
 - Validate number
 - Exception handling
 - Send email
- Common business functions which can be used in another business processes, e.g. in Student Management System:
 - Get list of students
 - Get student's information
- Shared objects or share classes:
 - Header class, footer class
 - Constant variables, language packages

Code Library

- Library of common functions
- Library of module, component, subsystem



Key points

- Advantages of reuse are lower costs, faster software development and lower risks.
- Design patterns are high-level abstractions that document successful design solutions.
- Program generators are also concerned with software reuse
 - the reusable concepts are embedded in a generator system.

TIME TO WAKE UP

